ChatGPT

HESPERIA KAN Feasibility Research and Concept-Layer Specification

Bottom-line conclusion

KAN is a good fit for HESPERIA only in a narrow, disciplined sense: not as a replacement for the engine, but as a way to turn a concept relation from a static scalar into an explicit transfer function on the edge. That is the durable scientific idea in KAN: learnable univariate functions live on edges instead of fixed activations living on nodes. The original KAN paper frames this as the core architectural shift, and the pykan authors themselves are explicit that KAN is aimed at smaller, science-oriented, interpretability-heavy settings rather than serving as a universal drop-in replacement for general machine-learning pipelines. [1]

On your five success criteria, the answer is mostly yes, with hard boundaries. KAN can provide meaningful semantic transfer functions; those functions can remain deterministic and explainable if they are frozen, bounded, versioned, and inspected as explicit semantic objects; they can improve relation quality wherever thresholds, saturation, inhibition, and non-monotonic effects matter; they can support mirror-thinking if backward reasoning is implemented as a first-class rule object rather than assumed to be the automatic inverse of the forward curve; and there is a realistic migration path because graph-oriented KAN work, inverse-form KAN work, and KA-FCM already show that replacing scalar edges with explicit edge functions is technically viable. [2]

The part to reject is the hype-heavy reading of KAN. HESPERIA does not need a neural runtime black box, latent semantic neurons, or online gradient updates inside the concept layer. What it needs is a stricter semantic object model: explicit edge functions, optional context moderators, explicit backward expectations, and audit artifacts good enough that a human can read the edge and understand what it does. That reading is far closer to KAN's symbolic/pruning workflow and KAN 2.0's symbolic/tree tooling than to "deep learning but with splines." [3]

KAN principles that matter for HESPERIA

The scientific core of KAN is straightforward. Classical MLPs use scalar weights on edges and fixed activation functions on nodes; KAN replaces those scalar edge weights with learnable univariate functions and keeps aggregation by summation. In the original KAN formulation, every weight is replaced by a spline-parameterized univariate function, and the authors argue this improves local interpretability because the learned functions can be visualized directly rather than inferred indirectly from distributed hidden-layer behavior. [4]

That matters for HESPERIA because your problem is not "which universal approximator is best," but "how do I make a relation legible as an object?" KAN 2.0 strengthens that direction by adding multiplication nodes, compilation of symbolic formulas into KANs, and tree conversion back out of networks into graph-like structures. Those are not just implementation extras; they show that the KAN family is most useful when it

becomes an interface between functional approximation and symbolic structure. That is exactly the pressure your concept layer is under. [5]

The pieces to import are therefore selective. Keep the idea of edge-local functions. Keep sparsification and pruning, because they provide a path to dead-edge detection and model simplification. Keep symbolic regression or symbolic distillation, because it offers a route from fitted curves to audit-ready formulas. Keep multiplication-like composition only when it is made explicit and named, because context interactions matter in semantic systems. [3]

The pieces to ignore are just as important. Schmidt-Hieber's analysis of the Kolmogorov-Arnold representation theorem is a useful warning: the theorem is mathematically deep, but it does not hand you a clean practical semantics for free, and some parts of the decomposition can behave wildly even when the target function is smooth. So the theorem should be treated as inspiration and loose justification, not as a literal proof that every learned edge in HESPERIA is automatically semantically meaningful. [6]

You should also ignore basis-family dogma. FastKAN shows that the original B-spline implementation can be closely approximated with Gaussian RBFs, and graph-KAN work evaluates both B-spline and RBF variants as viable graph-learning components. That means the durable principle is not "use B-splines because KAN used them first." The durable principle is "represent an edge as an explicit low-dimensional function." For HESPERIA, the most auditable basis should win, even if it is simpler than the best benchmarking choice in the literature. [7]

Compared with MLP-style thinking, the advantage is localization. An MLP distributes relation logic across hidden activations and many scalar parameters; a KAN-style edge says, in effect, "this is the shape of the influence from concept A to concept B." That is much closer to how semantic audit systems are supposed to work. But the cost is real: pykan warns about slow symbolic branches, tuning sensitivity, and smaller science-oriented task assumptions. So the right conclusion is not "KAN beats MLP." The right conclusion is "KAN contributes a better semantic edge primitive for the narrow class of problems where relation shape matters and auditability matters more than generic plug-and-play scale." [8]

Semantic transfer functions instead of static weights

A semantic relation can absolutely be represented as a transfer function instead of a static weight. In fact, this is the most defensible KAN-style interpretation for HESPERIA. If a current edge says only that Failure is linked to Shame with some strength, a transfer-function edge can say more: low activation may do little, mid-range activation may cross a threshold, high activation may saturate, and in some cases the relation may even reverse or bifurcate. KA-FCM is the closest direct precedent here: it replaces scalar causal weights with univariate spline functions on graph edges and shows that a single edge can model non-monotonic behavior without adding hidden layers or increasing graph density. [9]

That said, one hard correction is necessary. A plain KAN edge is **univariate**. So your example "Failure + social exposure + identity threat" is not honestly representable as one classical KAN edge if all three variables are meant to interact inside the same transformation. The clean options are either additive convergence at the target concept or an explicit interaction object such as `Failure × SocialExposure` or `Failure × IdentityThreat`, which is exactly the kind of extension KAN 2.0 introduces through

multiplication nodes. In other words, “Failure → Shame” can be a KAN-style transfer edge, but “Failure under social exposure” needs either moderators or an interaction node. [10]

A good HESPERIA formulation is therefore this:

$$c_{A\to B} = s_e \cdot f_e(x_A) \cdot \prod_{m\in M_e} \gamma_{e,m}(x_m)$$

where x_A is source activation, f_e is the main semantic transfer function, s_e is baseline signed strength, and $\gamma_{e,m}$ are optional moderator gates for explicitly named context concepts. This stays faithful to the edge-function spirit of KAN, while using MultKAN-style composition only where context interaction is really needed. [11]

That gives you a clean concept-layer mapping: KAN node becomes a named concept; KAN edge function becomes a semantic transfer function; KAN summation becomes target-side evidence aggregation; MultKAN multiplication becomes an explicit moderator interaction object; pruning becomes dead-edge simplification; symbolic regression and tree conversion become audit extraction tools; and unlabeled hidden neurons should be rejected unless they are converted into named, inspectable concepts. This is the concept-layer version of KAN that keeps the useful science and drops the black-box baggage. [12]

The relation taxonomy matters. Directional semantic relations such as `activates`, `opposes`, `expresses`, or `defends_against` are good candidates for transfer functions because they imply some kind of influence or modulation. Relations like `co_occurs_with` and often `observed_in` are different: they are usually associative or evidential rather than transformational. Those relations should often remain weighted support links or conditional evidence links unless you can state a genuine functional dependence. Otherwise you are forcing “function semantics” onto edges that do not actually mean “A transforms into B.” That is a design inference, but it follows directly from how KAN-style objects represent functions rather than bare adjacency. [13]

Canonical HESPERIA semantic edge object

The right edge object for HESPERIA is not a neural parameter blob. It is a typed, versioned semantic artifact that stores forward behavior, backward expectations, context moderation, and audit provenance as first-class fields. KAN’s value here is that it licenses the move from scalar weights to explicit edge functions; KAN 2.0 and pykan add the stronger point that such functions can be simplified, pruned, and in some cases translated into symbolic forms. [14]

```
edge_id: EDGE.FAILURE.SHAME.v1
source_concept: FAILURE
target_concept: SHAME
relation_type: activates

strength:
  sign: positive
  baseline: 0.62
  confidence: 0.84
```

```
transfer_function:
  family: piecewise_cubic
  domain: [0.0, 1.0]
  codomain: [0.0, 1.0]
  parameters:
    knots: [0.0, 0.2, 0.5, 0.8, 1.0]
    coefficients: [...]
  operating_range: [0.0, 0.9]
  shape_constraints:
    - thresholded
    - saturating
  symbolic_form: null

moderator_clauses:
  - concept: SOCIAL_EXPOSURE
    mode: multiplicative_gate
    function_ref: GATE.EXPOSURE.SHAME.v1
  - concept: IDENTITY_THREAT
    mode: reroute_bias
    target_override: HUMILIATION

forward_rule:
  aggregation: sum_then_clamp
  contribution_trace: true

backward_rule:
  mode: explicit_inverse_branches
  inverse_branches:
    - source_band: [0.35, 0.65]
      expected_contribution: moderate
      confidence: 0.70
    - source_band: [0.65, 1.00]
      expected_contribution: strong
      confidence: 0.85

expected_causes:
  - FAILURE
  - REJECTION
  - STATUS_LOSS

constraints:
  deterministic_runtime: true
  online_learning: false
  max_complexity: low
  monotonicity_required: false

audit_metadata:
```

```
  version: 1
  provenance: curated_plus_offline_fit
  calibration_basis: session_logs_v3
  reviewer: semantic_audit_board
  tests_passed:
    - bounds_check
    - shape_check
    - inverse_check
    - reproducibility_check
  checksum: sha256:...
```

`source_concept`, `target_concept`, and `relation_type` preserve the existing graph contract, which matters because HESPERIA already has concept nodes and typed relations. `strength` remains necessary even after introducing a curve, because relation trust, confidence, and baseline polarity are not the same thing as relation shape. You still need a compact signed prior that says whether the edge is generally excitatory, inhibitory, weak, or uncertain. [13]

`transfer_function` is the heart of the KAN adaptation. It should contain a **named function family**, explicit parameters, a bounded domain and codomain, an operating range, and optional symbolic form. This is what makes the edge inspectable. KA-FCM's use of local-support splines is relevant because local support makes edge behavior analyzable segment by segment, and KAN 2.0 plus pykan's symbolic workflow are relevant because they show a practical path from flexible fitted functions to human-readable artifacts. For HESPERIA, the hard rule should be that the function parameters are named coefficients or knots, never opaque tensors buried in model internals. [15]

`moderator_clauses` is the field missing from your draft that matters most. Without it, you are forced either to lie about classical KAN edges being multivariate or to hide context in a latent layer. The clean answer is to keep the primary edge univariate and add explicit moderator clauses or interaction objects when social exposure, identity threat, timing, or any other named context concept changes the transfer shape. That is faithful to the KAN idea and much safer than promoting hidden semantic nodes. [16]

`forward_rule` and `backward_rule` should be separate fields, not mirror images of one another. Forward propagation answers "given active causes, what effects should appear?" Backward propagation answers "given this observed effect, what upstream causes would make sense?" KINN matters here because it shows KAN-style systems can support inverse-form reasoning, but inverse reasoning is still a different object from the forward map. In semantic graphs, that difference is even more important because many relations are non-monotonic or many-to-one, so a backward rule is often a ranked set of inverse branches rather than a clean mathematical inverse. [17]

`expected_causes`, `constraints`, and `audit_metadata` are what make the edge suitable for HESPERIA rather than just interesting in a notebook. `expected_causes` supports mirror propagation and abductive ranking; `constraints` forbids the system from drifting into online self-modification or unchecked complexity; `audit_metadata` makes every relation versionable, reproducible, and reviewable. This part is an engineering synthesis, but it is directly grounded in KAN's explicit edge functions, pykan's pruning/symbolic workflow, and the science-oriented KAN emphasis on legibility over raw black-box throughput. [18]

Mirror propagation and semantic error correction

A bidirectional concept architecture is feasible, but it should not be sold as “backpropagation” in the neural-network sense. The better model is **explanatory mirror propagation**. Forward propagation remains ordinary concept-to-concept influence through explicit transfer functions. Backward propagation becomes expectation reconstruction: when a target concept is active, incoming edges use their backward rules to produce a ranked expectation of which causes should be present, at what bands, and with what confidence. KAN 2.0’s bidirectional science-to-KAN / KAN-to-science framing and KINN’s use of inverse-form reasoning both support this direction, even though they are solving different domain problems. [19]

In practical terms, the mirror pass can be organized around three quantities:

$$\hat{x}_B = \mathrm{clamp}\left(\sum_{e:i\to B} c_e\right), \qquad r_B = x_B^{obs} - \hat{x}_B, \qquad g_{A|B} = E(A \mid B) - x_A^{obs}$$

Here $\hat{x}_B$ is the forward-predicted activation, r_B is the target residual, and $g_{A|B}$ is the cause-gap for a candidate upstream concept. This turns mirror propagation into a model-based diagnostic layer: the system compares what the graph predicted, what actually appeared, and which causes are missing or unexpectedly absent. That is not copied from one paper verbatim, but it is the natural semantic generalization of explicit edge functions plus inverse-form reasoning. [20]

This is where Q5 becomes concrete. Dead concepts become detectable when a concept and its incident edge functions spend most of their observed operating range in flat or irrelevant regions, or when pruning repeatedly removes their practical contribution. False activations become visible when a target is active but its backward-expectation profile is unsupported by the upstream state. Overactive concepts appear when one source repeatedly drives too many downstream targets into saturation. Contradiction routes appear when simultaneous paths imply mutually incompatible states. Missing semantic bridges appear when residuals recur in the same local subgraph and the known upstream causes systematically fail to account for the observed target. I infer these diagnostics from the combination of explicit edge shape, pruning, and graph-level functional reasoning. [21]

That does amount to a form of semantic error correction, but only in the narrow, auditable sense. The correction should be “flag, explain, and propose repair,” not “quietly update the live concept graph.” For example, if `WITHDRAWAL` is active and the backward rules strongly expect `SHAME`, `FEAR`, `REJECTION`, or `EXHAUSTION`, but only weak upstream support exists, the system can emit an under-explained target alert, show the missing-cause gaps, and suggest a candidate missing bridge or malformed edge function. That is a strong mirror-thinking mechanism, and it stays compatible with deterministic audit because the live runtime remains frozen while residuals become review signals. [22]

Feasibility classification and migration path

Immediately useful. The first category is clear: add an optional `transfer_function` field to selected directional concept relations; support a small family of bounded, human-readable transfer shapes such as thresholded, saturating, inhibitory, and single-peak/non-monotonic; add per-edge contribution traces; and store an explicit `backward_rule` plus ranked `expected_causes`. This is the part of KAN that maps

cleanly onto HESPERIA right now, because it uses edge-local functions, preserves graph form, and aligns with audit-oriented symbolic workflows. [23]

Useful after concept-graph expansion. The next category needs more semantic structure first. MultKAN-style moderator interactions are useful, but only when HESPERIA has enough named context concepts to justify them. Offline fitting from audited activation logs is useful, but only when you can distill the result back into a bounded spline or symbolic rule and review it before deployment. Residual-based missing-bridge detection is useful, but only when relation types and contradiction semantics are rich enough that the repair suggestions mean something. This is realistic, but it depends on graph maturity and disciplined audit tooling. [24]

Theoretical but impractical. A full end-to-end learned KAN over the entire concept layer, with arbitrary interaction depth, automatic inversion of every edge, and online self-updating functions, is the wrong direction. The theorem itself does not guarantee tame or uniquely meaningful decompositions, pykan's own notes emphasize efficiency limits and tuning sensitivity, and the symbolic branch is slow enough that even the official tooling warns about it. That kind of whole-graph neuro-symbolic ambition would almost certainly erode the determinism and inspectability you are trying to preserve. [25]

Should be rejected. Four ideas should be discarded outright for the concept layer: using KAN as a replacement for HESPERIA's ontology/propagation/audit stack; introducing unlabeled hidden semantic neurons; treating purely descriptive relations as if they were causal transfer functions by default; and allowing runtime online learning to mutate edge behavior during normal operation. Even the KAN authors warn that KANs and MLPs do not simply replace one another, and the closest graph-causal analogue, KA-FCM, argues for richer edge behavior **without** hidden-layer detours when interpretability is the goal. [26]

The realistic migration path is therefore conservative and practical. Keep the current weighted relation as the baseline edge contract. Add `transfer_function` as an optional extension on a pilot set of high-value directional relations. Represent those functions first as hand-authored or manually calibrated piecewise objects. Add a backward expectation rule per pilot relation. Only after that should you fit curves offline from audited logs, prune them, and distill them into symbolic or piecewise-canonical forms. In runtime, freeze everything: no training, no hidden latent semantics, no silent graph edits. That path keeps HESPERIA recognizably HESPERIA while borrowing exactly the part of KAN that is worth borrowing. [27]

The blunt verdict is this: **adopt KAN principles, not KAN as a product architecture.** The scientifically useful import is the semantic transfer-function edge. The right deliverable for HESPERIA is a deterministic, inspectable, bidirectional semantic-edge object with explicit forward shape, backward expectation, moderator clauses, and audit metadata. Done that way, KAN is not a threat to HESPERIA's determinism. It is a way to make relation semantics finally say not only **that** two concepts are linked, but **how** one concept tends to become, amplify, suppress, or reroute another. [23]

---

[1] [2] [4] [9] [10] [11] [13] [14] [18] [20] [23] https://arxiv.org/abs/2404.19756
https://arxiv.org/abs/2404.19756

[3] [8] [26] [27] https://github.com/kindxiaoming/pykan
https://github.com/kindxiaoming/pykan

5 12 16 19 24 https://arxiv.org/abs/2408.10205
https://arxiv.org/abs/2408.10205

6 25 https://arxiv.org/abs/2007.15884
https://arxiv.org/abs/2007.15884

7 https://arxiv.org/html/2405.06721v1
https://arxiv.org/html/2405.06721v1

15 21 https://arxiv.org/html/2604.05136v1
https://arxiv.org/html/2604.05136v1

17 22 https://arxiv.org/abs/2406.11045
https://arxiv.org/abs/2406.11045